

《软件过程改进方案设计文档》

刘瑞雪 2022141090116

一、设计原则

原则	说明
可落地性	选用成熟工具，避免纯理论构想
低侵入性	在不推翻原有过程框架的前提下融入AI节点
可度量性	每个AI介入点都定义量化评估指标
风险可控	建立AI输出的质量验证机制

二、新增AI相关角色

2.1 角色定义

在原有角色体系（项目经理、需求人员、设计人员、编码人员、测试人员、QA、配置管理）基础上，新增以下AI相关角色：

角色	人数	职能描述	对应阶段
AI训练师	1 (兼任测试人员)	负责AI工具的Prompt工程优化、 测试用例模板设计、AI输出质量评估	测试验证、 编码实现
AI测试分析师	1 (兼任测试人员)	负责AI生成测试用例的审核与验证、 缺陷预测结果分析	测试验证
AI编码助手管理员	1 (兼任编码人员)	负责GitHub Copilot等工具的配置管理、 代码生成质量监控	编码实现

2.2 更新后的责任矩阵（AI相关活动）

AI相关活动	项目经理	编码人员	测试人员	AI训练师	AI测试分析师	AI编码助手管理员
AI工具选型与配置	A	C	C	R	-	C
Prompt工程优化	-	-	-	R	C	-
AI生成代码审核	-	R	-	C	-	C

AI相关活动	项目经理	编码人员	测试人员	AI训练师	AI测试分析师	AI编码助手管理员
AI生成用例验证	-	-	R	-	C	-
AI输出质量标准定义	A	C	C	R	R	C

注：A=审批，R=负责，C=参与

三、软件过程重构方案

3.1 整体过程改进框架

以下为HomePick项目改进后的软件过程全景图（Mermaid格式）：

需求分析阶段

用户调研

AI辅助需求文档生成

需求评审

需求基线锁定

设计阶段

架构选型

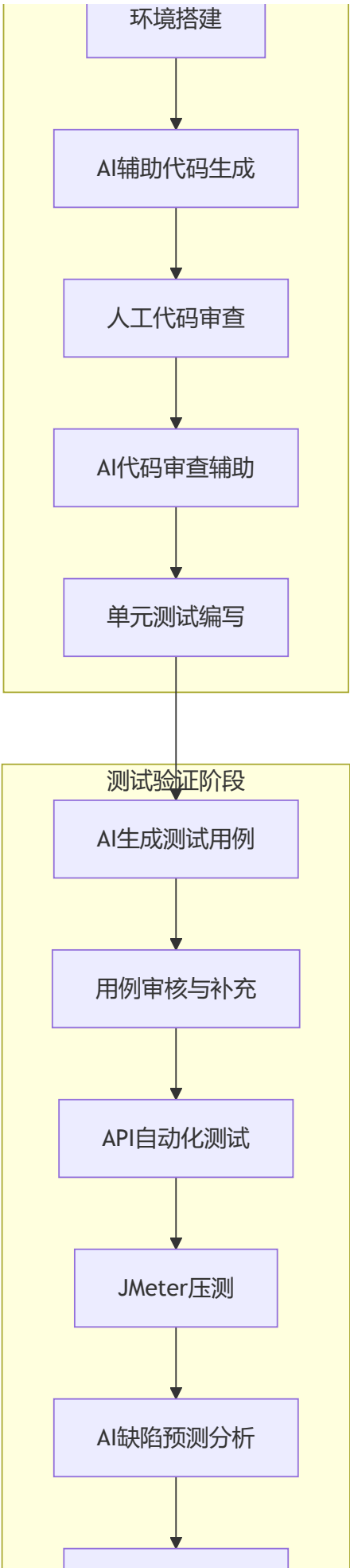
数据库设计

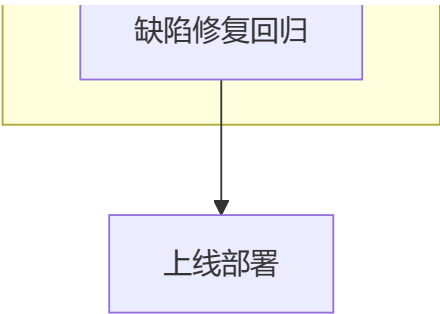
缓存策略设计

秒杀流程设计

设计评审

编码实现阶段



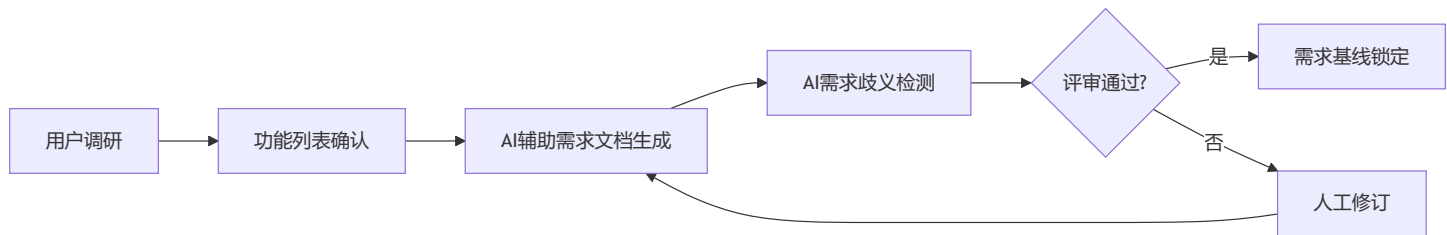


3.2 需求分析阶段改进细节

3.2.1 原流程

用户调研 → 功能列表确认 → 技术可行性评审 → 需求基线锁定

3.2.2 改进后流程



3.2.3 AI介入点说明

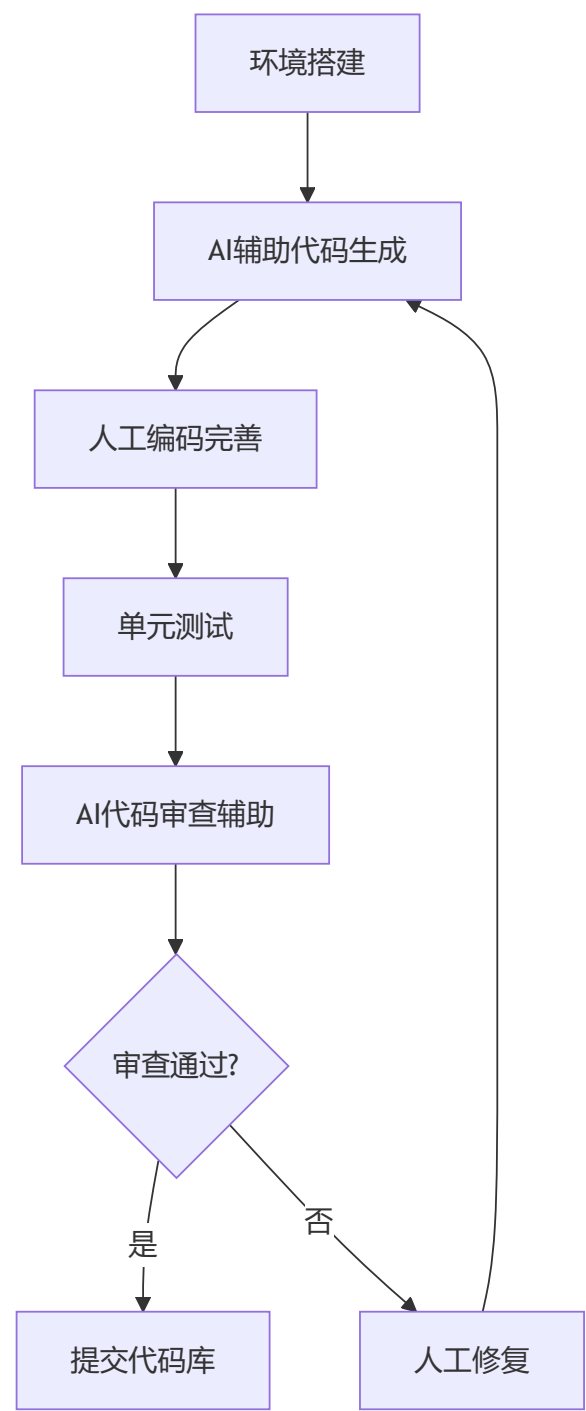
AI介入点	工具	输入	输出	人工参与
需求文档生成	LLM (DeepSeek)	功能列表、 用户故事	结构化需求文档 (Markdown)	需求人员审核
需求歧义检测	LLM	需求文档草稿	歧义点标注、 完整性检查报告	需求人员确认修订

3.3 编码实现阶段改进细节

3.3.1 原流程

环境搭建 → 缓存模块实现 → 秒杀模块实现 → MQ模块实现 → 单元测试 → 代码审查

3.3.2 改进后流程



3.3.3 AI介入点说明

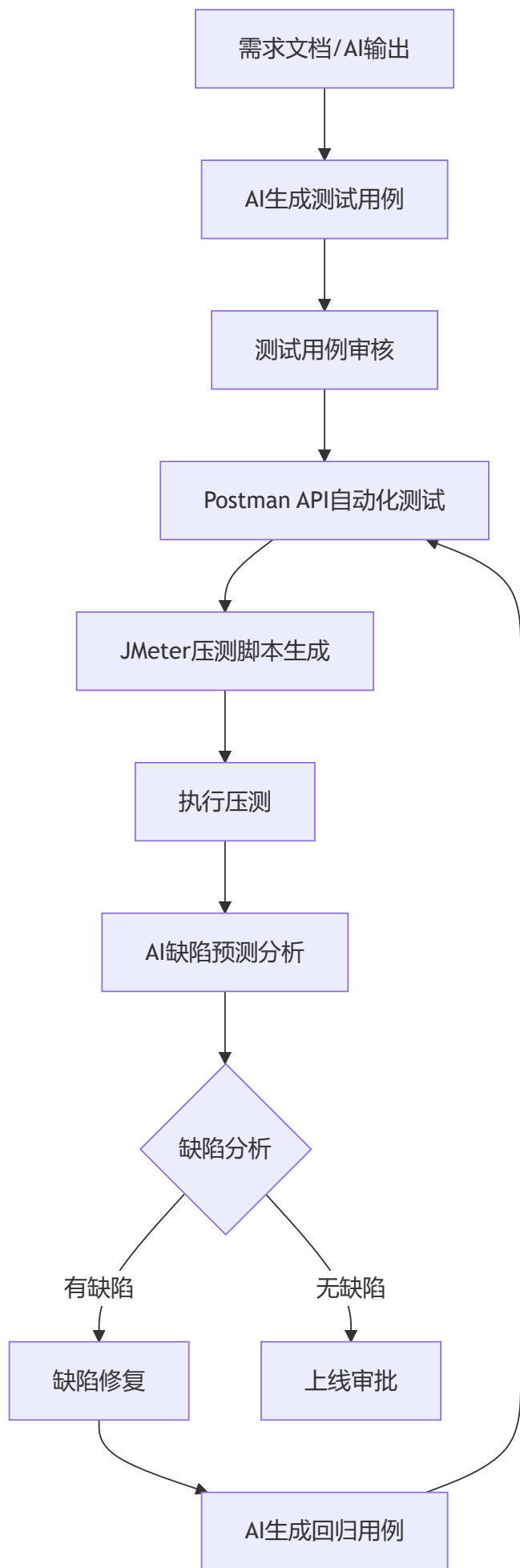
AI介入点	工具	输入	输出	预期提效
代码生成	GitHub Copilot	自然语言注释/ 函数签名	代码建议/完整函数	编码效率提升30%
代码审查	GitHub Copilot + SonarQube AI	PR代码变更	代码质量报告、 安全漏洞标注	审查时间减少40%

3.4 测试验证阶段改进细节

3.4.1 原流程

Postman API测试 → JMeter压测 → 缺陷修复 → 回归测试 → 上线审批

3.4.2 改进后流程



3.4.3 AI介入点说明

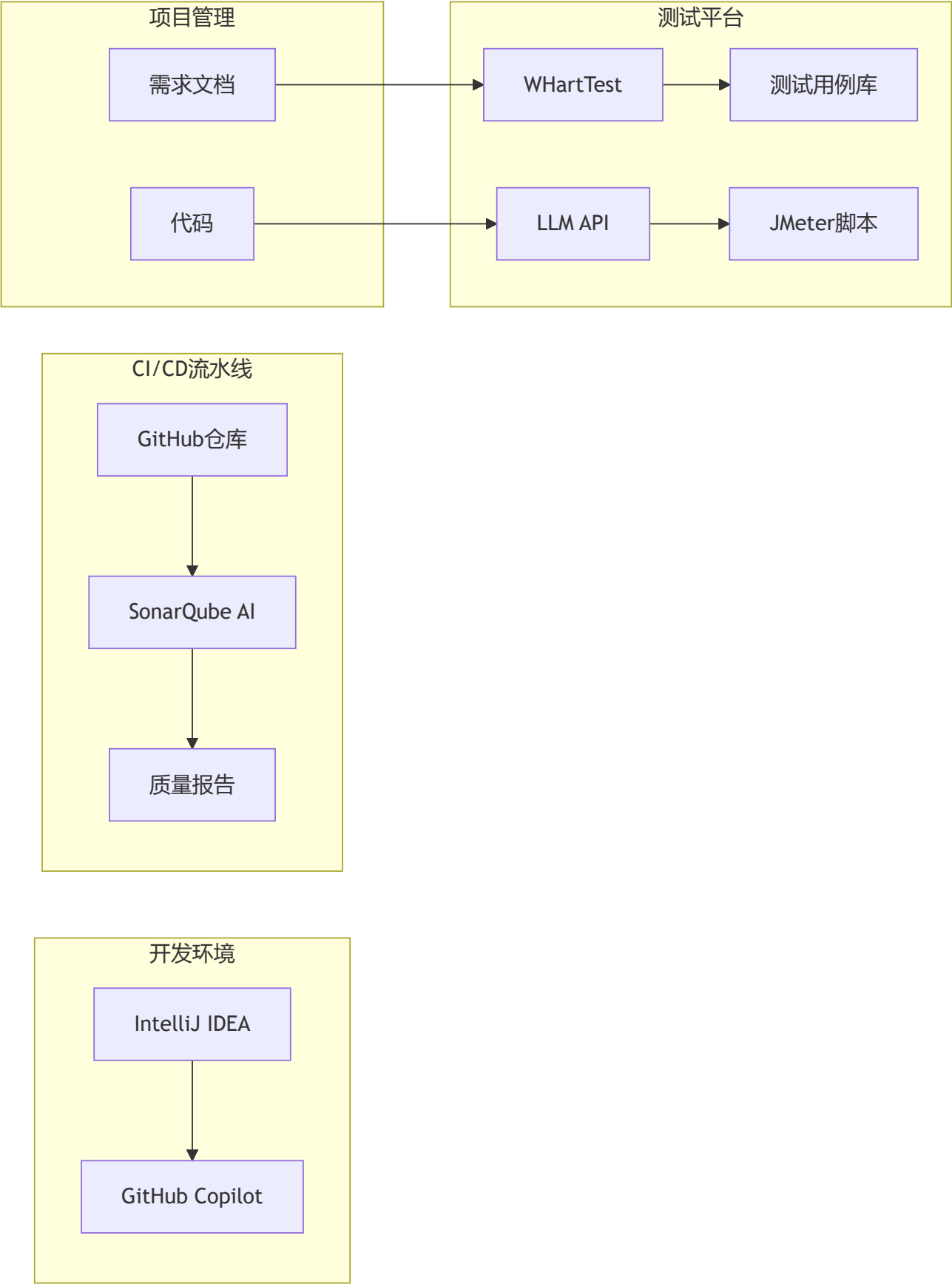
AI介入点	工具	输入	输出	预期提效
测试用例生成	WHartTest / LLM	需求文档、API定义	单元测试用例、接口测试脚本	用例编写时间减少60%
压测脚本生成	LLM	JMeter模板、API信息	JMeter压测脚本（JMX）	脚本编写时间减少50%
缺陷预测	LLM + 历史数据	代码变更、测试结果	缺陷概率分布、高危模块标注	缺陷发现时间提前30%
回归用例生成	LLM	缺陷报告、修复记录	针对性回归测试用例	回归测试效率提升40%

四、工具链选型与集成

4.1 工具清单

改进环节	工具名称	核心功能	集成方式
需求文档生成	DeepSeek / 通义千问	自然语言生成结构化需求文档	Web API / 本地部署
代码生成	GitHub Copilot	实时代码建议、函数生成	IDE插件（IntelliJ IDEA/VSCode）
代码审查	GitHub Copilot + SonarQube	代码质量分析、安全扫描	CI/CD流水线集成
测试用例生成	WHartTest	AI驱动的用例自动生成	Web平台 / API调用
压测脚本生成	DeepSeek	从API定义生成JMeter脚本	Web API
缺陷预测	自建Prompt + LLM	基于代码变更预测缺陷	API调用

4.2 工具集成架构图



4.3 预期效率提升指标

改进场景	改进前耗时	改进后预估耗时	提升幅度
需求文档编写（单模块）	2小时	1小时	50%
单元测试用例编写（单模块）	3小时	1.5小时	50%
接口测试脚本编写（单接口）	30分钟	10分钟	67%
JMeter压测脚本编写	2小时	45分钟	63%
代码审查（PR级别）	1小时	30分钟	50%

综合预期：整体测试验证阶段效率提升约40%-50%，编码阶段效率提升约30%。

五、风险评估与应对策略

5.1 风险识别矩阵

风险类别	具体风险	发生概率	影响程度	风险等级
数据安全	代码/需求数据泄露至AI服务商	中	高	高
模型偏见	AI生成内容存在偏见或错误逻辑	高	中	中
技术依赖	过度依赖AI导致团队能力退化	中	中	中
质量风险	AI生成代码/用例质量不达标	高	高	高
合规风险	AI生成代码的版权归属不清	低	高	中
成本风险	AI工具API调用费用超预期	低	低	低

5.2 应对策略

5.2.1 数据安全应对

策略	具体措施
数据脱敏	调用AI API前，移除代码中的敏感信息（密钥、用户数据）
本地化部署	优先选择支持本地部署的工具（如本地LLM）
工具选型	选用通过安全认证的企业版工具（如GitHub Copilot Business）

5.2.2 质量风险应对

策略	具体措施
人工审核机制	AI生成内容必须经过人工审核，审核人签字确认
质量标准定义	建立AI输出物质量标准（见6.2节）
自动化验证	AI生成代码必须通过单元测试和静态分析

5.2.3 技术依赖应对

策略	具体措施
轮岗机制	团队成员定期在没有AI工具的情况下完成开发任务
知识沉淀	要求AI生成的代码必须附有人工注释说明
工具备份	准备替代工具方案，避免单一工具不可用导致阻塞

5.3 应急预案

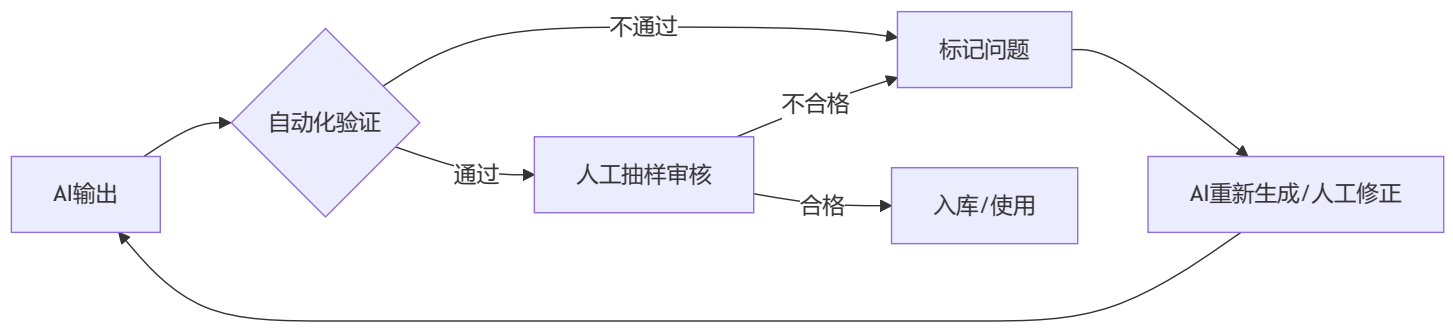
风险场景	应急响应措施	响应时效
AI服务不可用	切换至备用工具（如从GitHub Copilot切换至通义灵码）	30分钟内
AI输出质量严重偏差	回退至纯人工模式，同时提交问题反馈至工具厂商	立即
数据泄露事件	立即停止使用相关工具，按组织安全流程上报	1小时内

六、AI输出物的质量标准与验证机制

6.1 质量标准定义

AI输出物	质量标准	验证方式	合格阈值
需求文档	功能覆盖率100%、无歧义描述	人工评审 + AI检测	评审通过率≥95%
生成代码	通过静态分析、符合项目编码规范	SonarQube扫描	阻断问题=0
测试用例	分支覆盖率≥80%、用例可执行	人工审核 + 自动化执行	通过率≥90%
缺陷预测	预测准确率、召回率	与实际发现的缺陷对比	准确率≥60%

6.2 AI输出物验证流程



6.3 AI贡献度度量指标

指标	定义	采集方式	目标
AI生成代码占比	AI辅助生成代码行数/总代码行数	IDE插件统计	≥25%
AI测试用例生成率	AI生成用例数/总用例数	测试平台统计	≥40%
AI用例通过率	AI生成用例首次执行通过比例	自动化测试报告	≥85%
AI辅助审查覆盖率	AI辅助审查的PR占比	CI/CD统计	100%
缺陷逃逸率	上线后发现的缺陷/总缺陷	缺陷管理工具	≤10%

七、改进方案总结

7.1 核心改进点

- 1. 新增3个AI相关角色（AI训练师、AI测试分析师、AI编码助手管理员）
- 2. 重构3个阶段的活动流程（需求、编码、测试），融入AI执行节点
- 3. 选型6个AI工具，覆盖全生命周期关键环节
- 4. 建立AI输出物质量标准与验证机制，确保质量可控
- 5. 定义5项AI贡献度指标，实现过程可度量

7.2 预期收益

维度	改进前	改进后预期	提升幅度
需求文档编写效率	基线	+50%	提升
测试用例编写效率	基线	+60%	提升
代码审查效率	基线	+40%	提升
缺陷发现提前量	基线	+30%	提升

八、参考文献

1. 中国信通院. (2024). 《AI4SE 行业现状调查报告（2024年度）》.
2. 中国信通院. (2024). 《智能化软件工程技术和应用要求 第3部分:智能测试能力》（标准编号：AIIA/PG 0138-2024）.
3. Gtest全球软件测试技术峰会. (2025). 智能测试专场资料.
4. WHartTest官方文档. (2025). <https://mgdaaslab.github.io/WHartTest/>
5. Testin XAgent官方信息. (2025). <https://www.testin.net/xagent/>
6. GitHub Copilot官方文档. (2025). <https://docs.github.com/en/copilot>